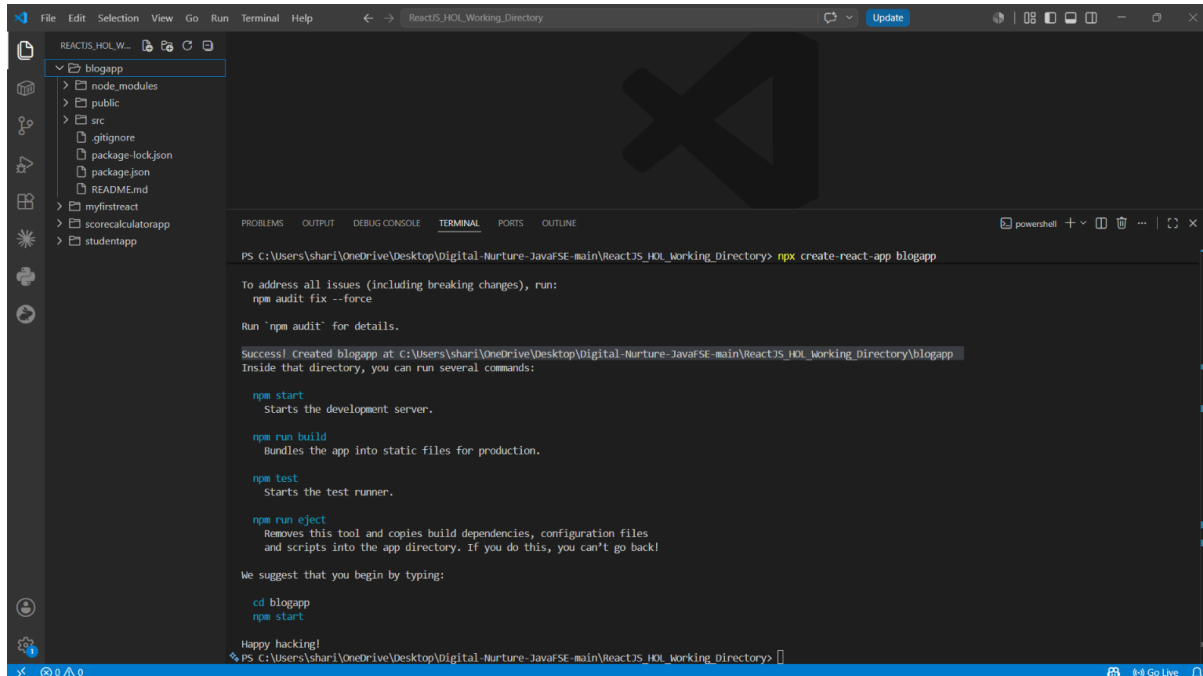


ReactJS Hands on Lab – 4

Creating the React project blogapp



The screenshot shows the Visual Studio Code interface with a file explorer on the left displaying the project structure. The main editor area shows the terminal output of the command `npm create-react-app blogapp`. The terminal text includes instructions on how to run the application, build it, and test it, along with a list of useful npm commands like `npm start`, `npm run build`, `npm test`, and `npm run eject`.

```
PS C:\Users\shari\OneDrive\Desktop\Digital-Nurture-JavaSE-main\ReactJS_HOI_working_Directory> npm create-react-app blogapp

To address all issues (including breaking changes), run:
  npm audit fix --force

Run 'npm audit' for details.

Success! Created blogapp at C:\Users\shari\OneDrive\Desktop\Digital-Nurture-JavaSE-main\ReactJS_HOI_working_Directory\blogapp
Inside that directory, you can run several commands:

  npm start
    Starts the development server.

  npm run build
    Bundles the app into static files for production.

  npm test
    Starts the test runner.

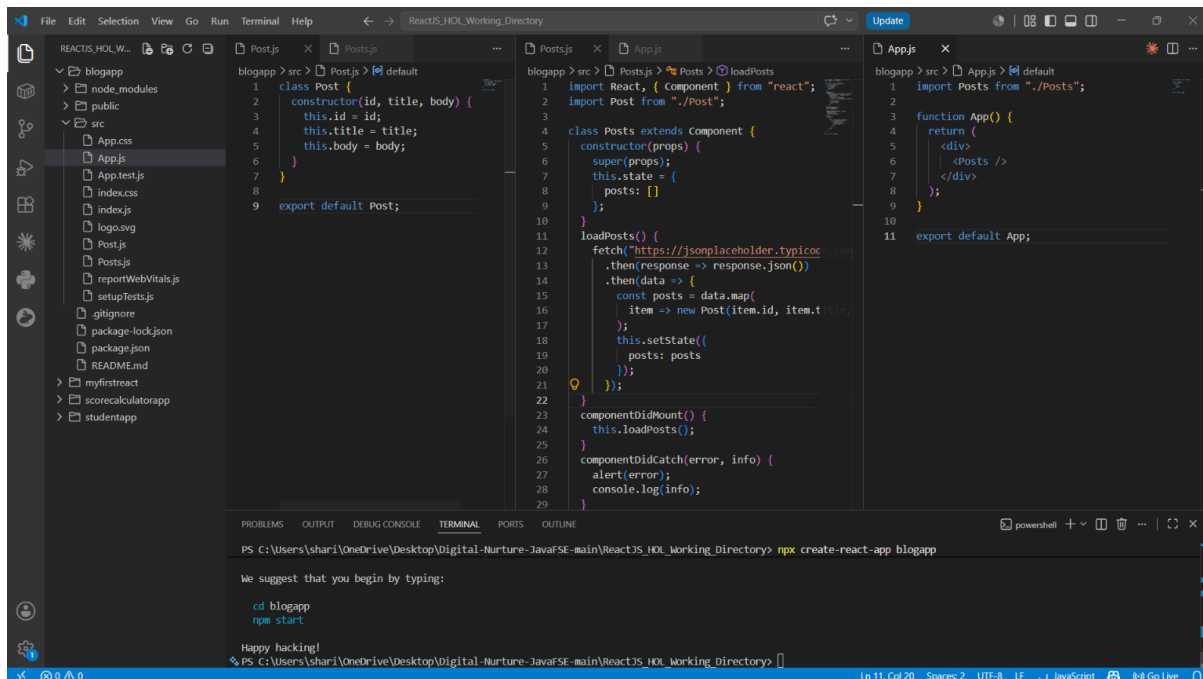
  npm run eject
    Removes this tool and copies build dependencies, configuration files
    and scripts into the app directory. If you do this, you can't go back!

We suggest that you begin by typing:

  cd blogapp
  npm start

Happy hacking!
PS C:\Users\shari\OneDrive\Desktop\Digital-Nurture-JavaSE-main\ReactJS_HOI_working_Directory>
```

Creating Post.js, Posts.js and App.js



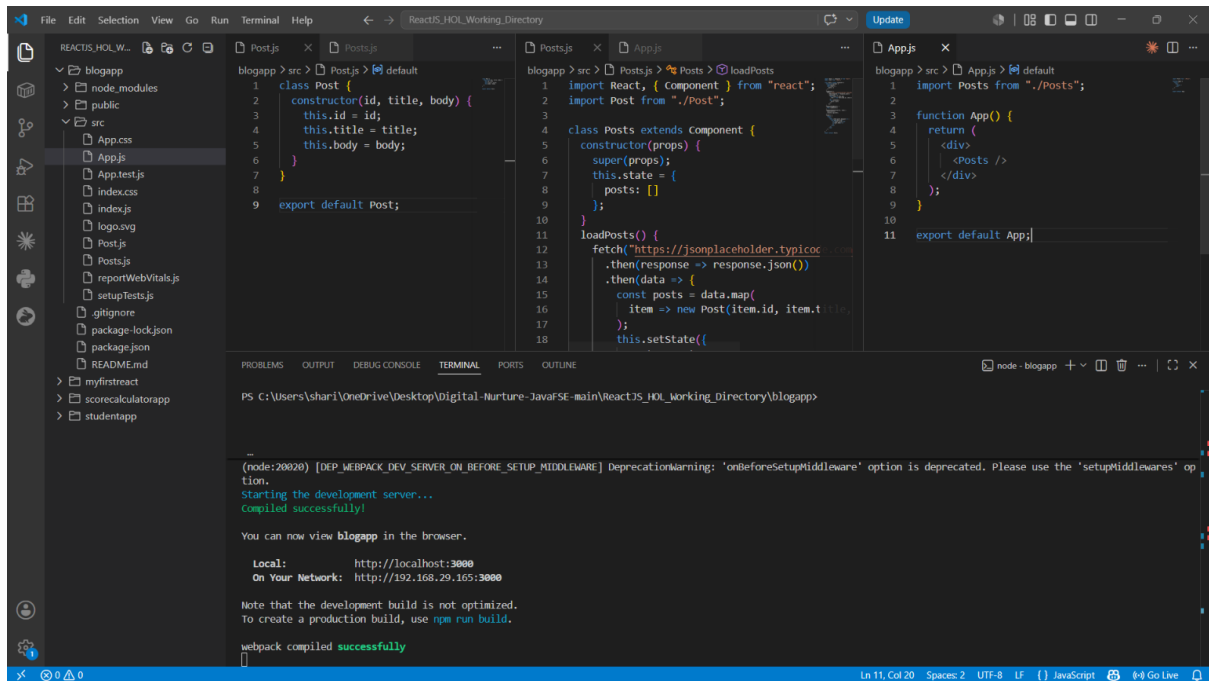
The screenshot shows the Visual Studio Code interface with three files open: `Post.js`, `Posts.js`, and `App.js`. The `Post.js` file contains a `Post` class component. The `Posts.js` file contains a `Posts` class component that extends `Component` and uses `loadPosts` to fetch data from a JSON placeholder. The `App.js` file contains an `App` function component that renders the `Posts` component.

```
blogapp > src > Posts.js > default
1 class Post {
2   constructor(id, title, body) {
3     this.id = id;
4     this.title = title;
5     this.body = body;
6   }
7 }
8
9 export default Post;

blogapp > src > Posts.js > loadPosts
1 import React, { Component } from "react";
2 import Post from "./Post";
3
4 class Posts extends Component {
5   constructor(props) {
6     super(props);
7     this.state = {
8       posts: []
9     };
10  }
11  loadPosts() {
12    fetch("https://jsonplaceholder.typicode.com/posts")
13      .then(response => response.json())
14      .then(data => {
15        const posts = data.map(
16          item => new Post(item.id, item.title, item.body)
17        );
18        this.setState({
19          posts: posts
20        });
21      });
22  }
23  componentDidMount() {
24    this.loadPosts();
25  }
26  componentDidCatch(error, info) {
27    alert(error);
28    console.log(info);
29  }
30 }

blogapp > src > App.js > default
1 import Posts from "./Posts";
2
3 function App() {
4   return (
5     <div>
6       <Posts />
7     </div>
8   );
9 }
10
11 export default App;
```

Starting the development Server



The screenshot shows the Visual Studio Code editor with a React application project. The file explorer on the left shows the project structure, including `blogapp`, `node_modules`, `public`, `src`, and `App.js`. The main editor displays the `App.js` file, which contains the following code:

```
1 import React, { Component } from "react";
2 import Post from "../Posts";
3
4 class App extends Component {
5   constructor(props) {
6     super(props);
7     this.state = {
8       posts: []
9     };
10  }
11  loadPosts() {
12    fetch("https://jsonplaceholder.typicode.com/posts")
13      .then(response => response.json())
14      .then(data => {
15        const posts = data.map(
16          item => new Post(item.id, item.title, item.body)
17        );
18        this.setState({
19          posts
20        });
21      });
22  }
23  render() {
24    return (
25      <div>
26        <h1>Blog App</h1>
27        <Posts />
28      </div>
29    );
30  }
31}
32
33export default App;
```

The terminal at the bottom shows the command `npm run dev` being executed, which starts the development server. The output indicates that the server is running on `http://localhost:3000` and that the application is now viewable in the browser.

OUTPUT

